

Tipos estructurados de datos I: Arrays, Cadenas y Estructuras

Dr. Paul Bustamante

Índice

- 1. Arrays**
- 2. Cadenas**
- 3. Estructuras**
- 4. Arrays de estructuras**
- 5. Uniones**
- 6. Ejemplos**

1. Arrays

Array (también conocido como *arreglo*, *vector* o *matriz*) es una colección de **variables relacionadas a las que se hace referencia por medio de un nombre en común**.

Es un **modo de manejar una gran cantidad de datos del mismo tipo bajo un mismo nombre o identificador**.

Su **forma general** es: **tipo nombre[tamaño]**;

Ejemplo: **double** datos[10];

En esta sentencia se reserva espacio para 10 variables de tipo **double**, las cuales se van a manejar por medio del nombre **datos** y un índice, el cual en C++ siempre empieza por “**cero**”.

Los elementos **se enumeran desde 0 hasta (n-1)**. Hay que tener mucho cuidado de no sobrepasar las dimensiones del **array**, en cuyo caso daría error el programa.

Si queremos acceder al primer elemento: **datos[0]=2.5;**

Al segundo: **datos[1]=4.5;**

Y así sucesivamente hasta el último valor: **datos[9]=3.5;**

Los elementos de un **array** se utilizan en las expresiones como cualquier variable.

Ejemplo: resultado = datos[3] * 5.0 / 2.0;

1. Arrays

Tal y como se hace con las variables, los **arrays** también pueden ser inicializados al momento de su creación:

Ejemplo:

```
int datos[5]={2, 4, -1, 7, 8};  
int datos[]={2, 4, -1, 7, 8};
```

Notas

- Tome nota que los valores van separados por una (,) y que al final de la llave (}) va un (;).
- Si faltan valores en la inicialización, pondrá ceros al resto.
- Si sobran, el compilador dará un mensaje de error.

Los **arrays** pueden ser de **una o varias dimensiones**:

Ejemplo:

```
double mat2x2[2][2];  
int aMatrix[3][2] = {{1, 4}, {2, 5}, {3, 6}};
```

El acceso a los elementos se hace de la misma forma que los de una dimensión.

Ejemplo:

```
resultado = mat2x2[0][1] * 2.5;
```

1. Arrays

Ejemplo de ordenar números dados por el teclado (usando array's)

```
#include <iostream>
using namespace std;
#define NUMMAX 30
void main ()
{
    int num, Max=0;
    int datos[NUMMAX];           //array de enteros
    cout << "Cuantos numeros desea ordenar (máximo 30)? ";
    cin >> num;
    if (num > NUMMAX) {
        cout << "Error.." << endl;
        return;
    }
    for (int i=0;i<num;i++){
        cout << "Dame el " << i+1 << " elemento: ";
        cin >> datos[i];
    }
    Max = datos[0];              //toma el primero
    for (int i=1;i<num;i++) {
        if (datos[i] > Max ) Max=datos[i];
    }
    //resultado
    cout << "Los numeros que has dado son: " << endl;
    for (int i=0;i<num;i++) cout << " " << datos[i];
    cout << "\nEl mayor es " << Max << endl;
}
```

Ejemplo 3-1

2. Cadenas

Las **cadenas** son **arrays de caracteres** (tipo **char**), **terminado en un carácter nulo**. Se pueden **inicializar** al momento de la declaración, si se desea, en cuyo caso se hará **usando las comillas**.

Ejemplo:

1. **char** ciudad[20]="San Sebastian";
2. **char** name[]="Juan Pablo";
3. **char** *direccion="Paseo Manuel Lardizabal 13";

(En la forma 2 y 3, el compilador reserva el número de caracteres para almacenar la información)

- Las cadenas **suelen contener texto** (nombres, frases, etc.), y éste se almacena en la parte inicial de la cadena. Para separar la parte de la cadena que contiene el texto de la parte no utilizada, se usa el *carácter fin de texto* que es el carácter nulo ('**\0**'). Este carácter se introduce automáticamente al inicializar la cadena.
- En una cadena **se puede acceder de forma independiente a cada elemento**, de la misma manera como se hace con los vectores, por ejemplo:

```
char name[20];  
name[0]='J'; name[1]='u'; name[2]='a'; name[3]='n'; name[4]='\0';  
cout << name;
```

2. Cadenas

Para la lectura de cadenas desde la consola, la sentencia ***cin*** presenta una dificultad: *no acepta los espacios ni las tabulaciones, corta las frases.*

Para poder capturar una frase completa (con espacios), se puede utilizar la función ***getline()*** de ***cin***, la cual forma parte de la biblioteca estándar del C++.

Su forma general es: ***cin.getline(nombre_de_cadena, max);***

```
#include <iostream>
using namespace std;

void main () {
    /* Distintas maneras de definir la cadena "name" */
    char name[5], name3[20];
    char name1[20] = "San Sebastian";
    char name2[] = "Juan Pablo";

    /* Introducción del nombre de distintas formas */
    name[0]='J'; name[1]='u'; name[2]='a'; name[3]='\n'; name[4]='\0';
    cout << "Introduce tu nombre y apellido: ";
    cin.getline(name3,20);

    /* Muestra los nombres introducidos de distintas formas */
    cout << name << endl << name1 << endl;
    cout << name2 << endl;
    cout << name3 << endl;
}
```

Ejemplo 3-2

2. Cadenas

- El siguiente ejemplo nos muestra el uso de **getline()** y **cin** con las cadenas. Así mismo podemos ver cómo acceder a cada uno de los elementos de la cadena.
- Se usa una de las funciones de la biblioteca del C++: **strlen()**, la cual nos devuelve la longitud de la cadena, sin considerar el carácter nulo ('\0');

```
#include <iostream>
using namespace std;
#include <string.h>           //para strlen()
void main (){
    char frase[80];

    cout << "Escriba una frase (<de 80 caract):";
    cin >> frase;
    cout << "Frase con cin:" << frase << endl;
    //con getline
    cout << "Escriba otra frase (<de 80 caract):"<<endl;
    cin.getlin(frase,80);
    cout << "Frase con gets():" << frase << endl <<endl;
    //imprimir los elementos, uno a uno
    int len=strlen(frase);
    for (int i=0;i<len;i++){
        cout << frase[i] << " <-> " << (int)frase[i] << endl;
    }
}
```

Ejemplo 3-3

3. Estructuras

Una estructura es una **forma de agrupar un conjunto de datos de distinto tipo** bajo un mismo nombre o identificador. Y se colocan fuera de la función *main*.

A cada uno de los datos de la estructura se le denomina **miembro**.

```
struct nombre{  
    tipo1 miembro1;  
    tipo2 miembro2;  
    tipon miembron;  
};
```

Ejemplo: Estructura que guarde los datos de un **alumno**: *Nombre, Apellidos, Carné y Edad*.

```
struct alumno{  
    char nombre[30];  
    char apellidos[80];  
    int edad;  
    unsigned long carnet;  
}; //no olvidar el ";" al final
```

La estructura crea un nuevo tipo de datos (en nuestro ejemplo es **alumno**). Para poder utilizar este nuevo tipo, hay que crear una nueva variable (tal como se hace con los tipos estándar de C++):

```
alumno alumno1, alumno2; //se crean dos variables
```

También se puede crear de forma directa las variables de una estructura, al declararla:

```
struct fecha{  
    short dia, mes, año;  
} dia1, dia2;
```

Se han creado de forma directa dos nuevas variables (**dia1** y **dia2**) del tipo **fecha**.

3. Estructuras

Para acceder a los miembros de una estructura, se utiliza el **operador punto (.)**, precedido por el nombre de la estructura y seguido por el nombre del miembro.

Ejemplo

```
struct alumno {
    char nombre [30];
    char apellido [80];
    int edad;
    unsigned long carnet;
};

void main () {
    alumno alumno1;
    cout << "Introduzca el nombre del alumno: ";
    //cin >> alumno1.nombre; // con esta orden el nombre sólo puede tener una
    //palabra
    cin.getline(alumno1.nombre,30);
    cout << "Introduzca los apellidos del alumno: ";
    //cin.ignore (); // nos sirve para borrar el contenido de buffer y recoger
    //correctamente el nombre
    cin.getline(alumno1.apellido,80);
    cout << "Introduzca la edad del alumno: ";
    cin >> alumno1.edad;
    cout << "Introduzca el numero de carnet del alumno: ";
    cin >> alumno1.carnet;

    cout << "\nLos datos del alumno introducido son los siguientes: " << endl;
    cout << "Nombre: " << alumno1.nombre << endl;
    cout << "Primer apellido: " << alumno1.apellido << endl;
    cout << "Edad: " << alumno1.edad << endl;
    cout << "Numero de carnet: " << alumno1.carnet << endl << endl;
}
```

Ejemplo 3-4

3. Estructuras - arrays

Arrays de estructuras: También se pueden crear arrays de estructuras, de la misma forma que se hace con los otros tipos.

Ejemplo: crear un array para almacenar los datos de 300 alumnos:

```
alumno datos[300];  
datos[0].carnet=900111;  
datos[1].carnet=900112;  
...  
datos[299].carnet=900410;
```

Para acceder a los miembros del array de estructuras se hace de la misma que los tipos estándar, es decir con un índice entre los corchetes.

Ámbito de una estructura: Se refiere a que los nombres de los miembros de una estructura son locales a la estructura y no interfieren con nombres iguales de variables del mismo tipo que estén fuera de la estructura.

Ejemplo:

La variable **x1**, tipo float, no interfiere con la variable **x1**, tipo int, del programa, ya que se encuentra dentro del ámbito de la estructura y para acceder a ella se hace mediante el nombre de la nueva variable creada: **c1**.

```
struct complejo{  
    float x1,y1;  
} c1;  
int x1;  
c1.x1=2.5;  
x1=5;
```

Ejemplo 3-5

3. Estructuras - arrays

Ejemplo

Base de datos de los alumnos que cursan una asignatura

Ejemplo 3-6

```
#include <iostream>
#include <stdio.h>
using namespace std;

struct alumno {
    char nombre [30];
    int edad;
};

void main () {
    alumno alumnoInforII [3];

    cout << "Introduzca el nombre de tres alumnos" << endl;
    cout << "-----" << endl;

    for (int i=0; i<3; i++){
        cout << "Introduzca el nombre del alumno " << i+1 << ": ";
        cin.getline(alumnoInforII [i].nombre,30);
        cout << "Introduzca la edad del alumno " << i+1 << ": ";
        cin >> alumnoInforII [i].edad;
        cin.ignore();
    }

    cout << "\nLos datos de los alumnos son los siguientes: " << endl;

    for (int i=0; i<3; i++){
        cout<<"Alumno "<<i+1<<". Nombre: "<<alumnoInforII[i].nombre<<", edad:
        "<<alumnoInforII[i].edad<<endl;
    }
}
```

4. Uniones

Una **unión** es otro tipo de datos complejo o agregado (al igual que las estructuras), con la **particularidad** que **consta de una sola posición de memoria** que es compartida por dos o más variables. También utilizan un mismo nombre o identificador para agrupar dichos datos.

A las variables de una unión también se les denomina **miembros**.

```
union nombre
{
    tipo miembro1;
    tipo miembro2;
    ....
    tipo miembroN;
} variables;
```

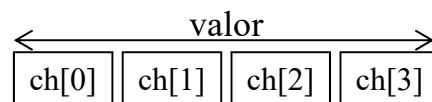


Ejemplo:

```
union dato{
    unsigned char ch[4];
    float valor;
}dat1;
```

Para acceder a los miembros de la unión, también se utiliza el **operador punto (.)**:

```
dat1.valor = 30.2525;
cout <<(int)dat1.ch[0] << " " <<(int)dat1.ch[3];
dat1.ch[3]=64;
cout << endl << dat1.valor << endl;
```



Cómo aparece la **unión** en la memoria.

Ejemplo: array de estructuras

Ejemplo

Base de datos de
películas (título y fecha)

```
#include <iostream>
using namespace std;

struct pelicula {
    char titulo[20];
    int fecha;
} films[3];

void main (void)
{
    int n;

    for (n=0; n<3; n++)
    {
        cout << "Introduzca el titulo: ";
        cin.getline(films[n].titulo,20);
        cout << "Introduzca year: ";
        cin >> films[n].fecha;
        cin.ignore();
    }
    cout << endl << "Ha introducido las siguientes peliculas:";
    for (n=0; n<3; n++)
    {
        cout << films[n].titulo << endl;
        cout << films[n].fecha << endl;
    }
}
```

Ejemplo 3-7